

*forwarding*

*-range*  
too subtle

<T

> is

Document #: P1870R1  
Date: 2019-11-08  
Project: Programming Language C++  
LEWG  
Reply-to: Barry Revzin  
<[barry.revzin@gmail.com](mailto:barry.revzin@gmail.com)>

## Contents

---

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Naming](#)
- [4 Opting into \*forwarding\* \*-range\*](#)
  - [4.1 History](#)
  - [4.2 Issues with overloading](#)
  - [4.3 How many mechanisms do we need?](#)
  - [4.4 Hard to get correct](#)
- [5 Proposal](#)
  - [5.1 Wording](#)
- [6 Acknowledgements](#)
- [7 References](#)

## § 1 Revision History

---

R0 [[P1870R0](#)] of this paper was presented to LEWG in Belfast. There was consensus to change the opt-in mechanism to use the trait rather than the non-member function as presented in the paper. However, there was unanimous dissent to remove the ability to invoke ranges `::begin` (and other CPOs) on rvalues. This draft adds that ability back.

In short, the only change then is the opt-in mechanism. All other functionality is preserved.

This paper also addresses the NB comments [US279](#) and [GB280](#), and is relevant to the resolution of [US276](#) and [US286](#).

## § 2 Introduction

---

One of the concepts introduced by Ranges is *forwarding* *-range*. The salient aspect of what makes a *forwarding* *-range* is stated in [\[range.range\]](#):

the validity of iterators obtained from the object denoted by E is not tied to the lifetime of that object.

clarified more in the subsequent note:

[ Note: Since the validity of iterators is not tied to the lifetime of an object whose type models forwarding -range, a function can accept arguments of such a type by value and return iterators obtained from it without danger of dangling. — end note ]

For example, `std::vector<T>` is not a forwarding -range because any iterator into a vector is of course dependent on the lifetime of the vector itself. On the other hand, `std::string_view` is a forwarding -range because it does not actually own anything - any iterator you get out of it has its lifetime tied to some other object entirely.

But while `span` and `subrange` each model forwarding -range, not all views do. For instance, `transform_view` would not because its iterators' validity would be tied to the unary function that is the actual transform. You could increment those iterators, but you couldn't dereference them. Likewise, `filter_view`'s iterator validity is going to be based on its predicate.

Really, a forwarding -range is quite a rare creature.<sup>1</sup>

Value category also plays into this. Notably, lvalue ranges all model forwarding -range – the “object” in question in this case is an lvalue reference, and the validity of iterators into a range are never going to be tied to the lifetime of some reference to that range. For instance, `std::vector<T>` is not a forwarding -range, but `std::vector<T>&` is. The only question is about rvalue ranges. If I have a function that either takes a range by forwarding reference or by value, I have to know what I can do with it.

Ranges uses this in two kinds of places:

- Many algorithms return iterators into a range. Those algorithms conditionally return either `iterator_t<R>` or dangling based on whether or not R satisfies forwarding -range (because if R did not, then such iterators would not be valid, so they are not returned). This type is called `safe_iterator_t<R>` and appears over 100 times in [\[algorithms\]](#).
- Range adapters can only be used on rvalue ranges if they satisfy either forwarding -range or they decay to a view. The former because you may need to keep iterators into them past their lifetime, and the latter because if you can cheaply copy it than that works too. This higher-level concept is called `viewable_range`, and every range adapter depends on it.

That is, forwarding -range is a very important concept. It is used practically everywhere. It also conveys a pretty subtle and very rare feature of a type: that its iterators can outlive it. Syntactically, there is no difference between a range, a view, and a forwarding -range, so the question is - how does a type declare itself to have this feature?

## § 3 Naming

The name forwarding -range is problematic. There is a concept `std::forward_range` which is completely unrelated. A fairly common first response is that it has something to do with forwarding iterators. But the name actually comes from the question of whether you can use a forwarding reference range safely.

However, coming up with a good name for it is very difficult. The concept has to refer to the range, but the salient aspect really has more to do with the iterators. Words that seem relevant are detachable, untethered, unfettered, nondangling. But then applying them to the range ends up being a mouthful: `range_with_detachable_iterators`. Granted, this concept isn't directly used in too many places so maybe a long name is fine.

The naming direction this proposal takes is to use the name `safe_range`, based on the existence of `safe_iterator` and `safe_subrange`. It still doesn't seem like a great name though, but at least all the relevant library things are similarly named.

Also the concept is still exposition-only, despite being a fairly important concept that people may want to use in their own code. This can be worked around:

```
template<class R>
concept my_forwarding_range = std::range<R>
    && std::same_as<std::safe_iterator_t<R>, std::iterator_t<R>>;
```

But this seems like the kind of thing the standard library should provide directly.

## § 4 Opting into *forwarding* -range

Types must opt into *forwarding* -range, and this is done by having non-member `begin` () and `end` () overloads that must take the type by either value or rvalue reference. At first glance, it might seem like this is impossible to do in the language but Ranges accomplishes this through the clever<sup>2</sup> use of poison-pill overload:

```
namespace __begin {
    // poison pill
    template <typename T> void begin(T&&) = delete;

    template <typename R>
    concept has_non_member = requires (R&& r) {
        begin(std::forward<R>(r));
    };
}

namespace N {
    struct my_vector { /* ... */ };
    auto begin(my_vector const&);
}
```

Does `N::my_vector` satisfy the concept `__begin::has_non_member`? It does not. The reason is that the poison pill candidate binds an rvalue reference to the argument while the ADL candidate binds an lvalue reference, and tiebreaker of rvalue reference to lvalue reference happens much earlier than non-template to template. The only way to have a better match than the poison pill for rvalues is to either have a function/function template that takes its argument by value or rvalue reference, or to have a function template that takes a constrained forwarding reference.

This is a pretty subtle design decision - why did we decide to use the existence of non-member overloads as the opt-in?

### § 4.1 History

This design comes from [stl2.547], with the expressed intent:

Redesign `begin/end` CPOs to eliminate deprecated behavior and to force range types to opt in to working with rvalues, thereby giving users a way to detect that, for a particular range type, iterator validity does not depend on the range's lifetime.

which led to [P0970R1], which describes the earlier problems with ranges `::begin` () thusly:

- 1 For the sake of compatibility with `std::begin` and ease of migration, `std::ranges::begin` accepted rvalues and treated them the same as `const lvalues`. This behavior was deprecated because it is fundamentally unsound: any iterator returned by such an overload is highly likely to dangle after the full-expression that contained the invocation of `begin`.
- 2 Another problem, and one that until recently seemed unrelated to the design of `begin`, was that algorithms that return iterators will wrap those iterators in `std::ranges::dangling` if the range passed to them is an rvalue. This ignores the fact that for some range types — `std::span`, `std::string_view`, and P0789's `subrange`, in particular — the iterator's validity does not depend on the range's lifetime at all. In the case where an rvalue of one of the above types is passed to an algorithm, returning a wrapped iterator is totally unnecessary.
- 3 The author believed that to fix the problem with `subrange` and `dangling` would require the addition of a new trait to give the authors of range types a way to say whether its iterators can safely outlive the range. That felt like a hack.

This paper was presented in Rapperswil 2018, partially jointly with [P0896R1], and as far as I can tell from the minutes, this subtlety was not discussed.

## § 4.2 Issues with overloading

In [stl2.592], Eric Niebler points out that the current wording has the non-member `begin()` and `end()` for `subrange` taking it by rvalue reference instead of by value, meaning that `const subrange` doesn't count as a *forwarding range*. But there is a potentially broader problem, which is that overload resolution will consider the `begin()` and `end()` functions for `subrange` even in contexts where they would be a worse match than the poison pill (i.e. they would involve conversions), and some of those contexts could lead to hard instantiation errors. So Eric suggests that the overload should be:

```
friend constexpr I begin(same_as<subrange> auto r) { return r.begin(); }
```

Of the types in the standard library that should model *forwarding range*, three of the four should take the same treatment (only `iota_view` doesn't need to worry). That is, in order to really ensure correctness by avoiding any potential hard instantiation errors, we have to write non-member `begin()` and `end()` function templates that constrain their argument via `same_as<R>?`

The issue goes on to further suggest that perhaps the current overload is really:

```
friend constexpr I begin(same-ish<subrange> auto&& r) { return r.begin(); }
```

And there is an NB comment that suggests the *same-ish* `<T>` `>` `auto` `&&` spelling for some times and `same_as` `<T>` `>` `auto` spelling for others. What's the distinction? To be honest, I do not understand.

Now we've started from needing a non-member `begin()`/`end()` that take an argument by value or rvalue reference – not necessarily to actually be invoked on an rvalue – but that runs into potential problems, that need to be solved by making that non-member a constrained template that either takes by value or forwarding reference, but constrained to a single type?

## § 4.3 How many mechanisms do we need?

At this point, we have three concepts in Ranges that have some sort of mechanism to opt-in/opt-out:

- *forwarding* *-range*: provide a non-member `begin()`/`end()` that take their argument by value or rvalue reference (but really probably a constrained function template)
- *view*: opt-in via the `enable_view` type trait
- *sized\_range*: opt-out via the `disable_sized_range` trait

I don't think we need different mechanisms for each trait. I know Eric and Casey viewed having to have a type trait as a hack, but it's a hack around not having a language mechanism to express opt-in (see also [P1900R0]). It's still the best hack we have, that's the easiest to understand, that's probably more compiler-efficient as well (overload resolution is expensive!)

## § 4.4 Hard to get correct

Now that MSVC's standard library implementation is open source, we can take a look at how they went about implementing the opt-in for *forwarding* *-range* in their implementation of `basic_string_view` [msvc.basic\_string\_view]:

```
#ifdef __cpp_lib_concepts
    _NODISCARD friend constexpr const_iterator begin(const basic_string_view& _Right)
noexcept {
    // non-member overload that accepts rvalues to model the exposition-only
    forwarding-range concept
    return _Right.begin();
}
    _NODISCARD friend constexpr const_iterator end(const basic_string_view& _Right)
noexcept {
    // Ditto modeling forwarding-range
    return _Right.end();
}
#endif // __cpp_lib_concepts
```

Note that these overloads take their arguments by reference-to-`const`. But the non-member overloads need to take their arguments by either value or rvalue reference, otherwise the poison pill is a better match, as described earlier. So at this moment, `std::string_view` fails to satisfy *forwarding* *-range*. If even Casey can make this mistake, how is anybody going to get it right?

## § 5 Proposal

The naming direction this proposal takes is to use the name `safe_range`, based on the existence of `safe_iterator` and `safe_subrange`. If a alternate name is preferred, the wording can simply be block replaced following the naming convention proposed in [P1871R0]. The proposal has four parts:

- Trait: introduce a new variable template `enable_safe_range`, with default value `false`.
- Concept: rename the concept *forwarding* *-range* to `safe_range`, make it non-exposition only, and have its definition be based on the type trait. Replace all uses of *forwarding* *-range* with `safe_range` as appropriate.
- CPO: Have ranges `::begin()` and ranges `::end()`, and their const and reverse cousins, check the trait `enable_safe_range` and *only* allow lvalues unless this trait is true.
- Library opt-in: Have the library types which currently opt-in to *forwarding* *-range* by providing non-member `begin` and `end` instead specialize `enable_safe_range`, and remove those overloads non-member

overloads.

## § 5.1 Wording

[ Editor's note: The paper P1664R1 opts `iota_view` into modeling what is now the `safe_range` concept by adding non-member `begin` `()` and `end` `()`. When we merge both papers together, `iota_view` should *not* have those non-member functions added. This paper adds the new opt-in by specializing `enable_safe_range`. ]

Change 21.4.1 [string.view.synop] to opt into `enable_safe_range`:

```
namespace std {
    // [string.view.template], class template basic_string_view
    template<class charT, class traits = char_traits<charT>>
    class basic_string_view;

    + template<class charT, class traits>
    + inline constexpr bool enable_safe_range<basic_string_view<charT, traits>>
= true;
}
```

Change 21.4.2 [string.view.template] to remove the non-member `begin/end` overloads that were the old opt-in:

```
template<class charT, class traits = char_traits<charT>>
class basic_string_view {

    - friend constexpr const_iterator begin(basic_string_view sv) noexcept {
return sv.begin(); }
    - friend constexpr const_iterator end(basic_string_view sv) noexcept { return
sv.end(); }

};
```

Change 22.7.2 [span.syn] to opt into `enable_safe_range`:

```
namespace std {
    // constants
    inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();

    // [views.span], class template span
    template<class ElementType, size_t Extent = dynamic_extent>
    class span;

    + template<class ElementType, size_t Extent>
    + inline constexpr bool enable_safe_range<span<ElementType, Extent>> = true;
}
```

Change 22.7.3.1 [span.overview] to remove the non-member `begin/end` overloads that were the old opt-in:

```
namespace std {
    template<class ElementType, size_t Extent = dynamic_extent>
    class span {
        [...]

    - friend constexpr iterator begin(span s) noexcept { return s.begin(); }
    - friend constexpr iterator end(span s) noexcept { return s.end(); }

    private:
        pointer data_; // exposition only
        index_type size_; // exposition only
    };
}
```

```

    template<class Container>
        span(const Container&) -> span<const typename Container::value_type>;
}

```

Change 24.2 [ranges.syn] to introduce the new trait and the new non-exposition-only concept:

```

#include <initializer_list>
#include <iterator>

namespace std::ranges {
    [ ... ]

    // [range.range], ranges
    template<class T>
        concept range = see below;

+   template <range T>
+   inline constexpr bool enable_safe_range = false;
+
+   template<class T>
+   concept safe_range = see below;

    [ ... ]

    // [range.subrange], sub-ranges
    enum class subrange_kind : bool { unsized, sized };

    template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind K
= see below>
        requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
        class subrange;

+   template<input_or_output_iterator I, sentinel_for<I> S, subrange_kind K>
+   inline constexpr bool enable_safe_range<subrange<I, S, K>> = true;

    // [range.dangling], dangling iterator handling
    struct dangling;

    template<range R>
-   using safe_iterator_t = conditional_t<forwarding-range<R>, iterator_t<R>,
dangling>;
+   using safe_iterator_t = conditional_t<safe_range<R>, iterator_t<R>,
dangling>;

    template<range R>
        using safe_subrange_t =
-   conditional_t<forwarding-range<R>, subrange<iterator_t<R>>, dangling>;
+   conditional_t<safe_range<R>, subrange<iterator_t<R>>, dangling>;

    // [range.empty], empty view
    template<class T>
        requires is_object_v<T>
        class empty_view;

+   template<class T>
+   inline constexpr bool enable_safe_range<empty_view<T>> = true;

    [ ... ]

```

```

// [range.iota], iota view
template<weakly_incrementable W, semiregular Bound = unreachable_sentinel_t>
    requires weakly-equality-comparable-with<W, Bound>
    class iota_view;

+ template<weakly_incrementable W, semiregular Bound>
+   inline constexpr bool enable_safe_range<iota_view<W, Bound>> = true;

[... ]

template<range R>
    requires is_object_v<R>
    class ref_view;

+ template<class T>
+   inline constexpr bool enable_safe_range<ref_view<T>> = true;

[... ]
}

```

Change the definitions of `ranges::begin()`, `ranges::end()`, and their `c` and `r` cousins, to only allow lvalues unless `enable_safe_range` is `true`, and then be indifferent to member vs non-member (see also [stl2.429]). The poison pill no longer needs to force an overload taking a value or rvalue reference, it now only needs to force ADL - see also [LWG3247]), but this change is not made in this paper.

Change. 24.3.1 [range.access.begin]:

- 1 The name `ranges::begin` denotes a customization point object. Given a subexpression `E` and an lvalue `t` that denotes the same object as `E`, if `E` is an rvalue and `enable_safe_range<remove_cvref_t<decltype((E))>> is false`, `ranges::begin(E)` is ill-formed. Otherwise, ~~The expression~~ `ranges::begin(E)` for some subexpression `E` is expression-equivalent to:
  - (1.1) — ~~`E`~~ ~~`±`~~ ~~`0` if `E` is a lvalue~~ ~~`+`~~ `0` if `t` is an lvalue of array type ([basic.compound]).
  - (1.2) — Otherwise, ~~if `E` is an lvalue~~, `decay-copy(E.t.begin())` if it is a valid expression and its type `I` models `input_or_output_iterator`.
  - (1.3) — Otherwise, `decay-copy(begin(E.t))` if it is a valid expression and its type `I` models `input_or_output_iterator` with overload resolution performed in a context that includes the declarations:
 

```

template<class T> void begin(T&&) = delete;
template<class T> void begin(initializer_list<T>&&) = delete;

```

 and does not include a declaration of `ranges::begin`.
  - (1.4) — Otherwise, `ranges::begin(E)` is ill-formed. [ Note: This case can result in substitution failure when `ranges::begin(E)` appears in the immediate context of a template instantiation. — *end note* ]
- 2 [ Note: Whenever `ranges::begin(E)` is a valid expression, its type models `input_or_output_iterator`. — *end note* ]

Change 24.3.2 [range.access.end] similarly:



1 The name `ranges::end` denotes a customization point object. Given a subexpression `E` and an lvalue `t` that denotes the same object as `E`, if `E` is an rvalue and `enable_safe_range <remove_cvref_t< decltype((E))>> is false`, `ranges::end(E)` is ill-formed. Otherwise, The expression `ranges::end(E)` for some subexpression `E` is expression-equivalent to:

- (1.1) — `E t + extent_v<T>` if `E` is an lvalue of array type ([basic.compound]) `T`.
- (1.2) — Otherwise, if `E` is an lvalue, `decay-copy(E t.end())` if it is a valid expression and its type `S` models `sentinel_for < decltype(ranges::begin(E))>`.
- (1.3) — Otherwise, `decay-copy(end(E t))` if it is a valid expression and its type `S` models `sentinel_for < decltype(ranges::begin(E))>` with overload resolution performed in a context that includes the declarations:

```
template<class T> void end(T&&) = delete;
template<class T> void end(initializer_list<T>&&) = delete;
```

and does not include a declaration of `ranges::end`.

- (1.4) — Otherwise, `ranges::end(E)` is ill-formed. [ *Note:* This case can result in substitution failure when `ranges::end(E)` appears in the immediate context of a template instantiation. — *end note* ]

2 [ *Note:* Whenever `ranges::end(E)` is a valid expression, the types `S` and `I` of `ranges::end(E)` and `ranges::begin(E)` model `sentinel_for <S, I>`. — *end note* ]

Change 24.3.5 [range.access.rbegin]:

1 The name `ranges::rbegin` denotes a customization point object. Given a subexpression `E` and an lvalue `t` that denotes the same object as `E`, if `E` is an rvalue and `enable_safe_range <remove_cvref_t< decltype((E))>> is false`, `ranges::rbegin(E)` is ill-formed. Otherwise, The expression `ranges::rbegin(E)` for some subexpression `E` is expression-equivalent to:

- (1.1) — If `E` is an lvalue, `decay-copy(E t.rbegin())` if it is a valid expression and its type `I` models `input_or_output_iterator`.
- (1.2) — Otherwise, `decay-copy(rbegin(E t))` if it is a valid expression and its type `I` models `input_or_output_iterator` with overload resolution performed in a context that includes the declaration:

```
template<class T> void rbegin(T&&) = delete;
```

and does not include a declaration of `ranges::rbegin`.

- (1.3) — Otherwise, `make_reverse_iterator(ranges::end(E t))` if both `ranges::begin(E t)` and `ranges::end(E t)` are valid expressions of the same type `I` which models `bidirectional_iterator` ([iterator.concept.bidir]).

- (1.4) — Otherwise, `ranges::rbegin(E)` is ill-formed. [ Note: This case can result in substitution failure when `ranges::rbegin(E)` appears in the immediate context of a template instantiation. — *end note* ]
- 2 [ Note: Whenever `ranges::rbegin(E)` is a valid expression, its type models `input_or_output_iterator`. — *end note* ]

Change 24.3.6 [range.access.rend]:

- 1 The name `ranges::rend` denotes a customization point object. Given a subexpression `E` and an lvalue `t` that denotes the same object as `E`, if `E` is an rvalue and `enable_safe_range <remove_cvref_t<decltype((E))>> is false`, `ranges::rend(E)` is ill-formed. Otherwise, The expression `ranges::rend(E)` for some subexpression `E` is expression-equivalent to:
- (1.1) — If `E` is an lvalue, `decay-copy(E.t.rend())` if it is a valid expression and its type `S` models `sentinel_for <decltype(ranges::rbegin(E))>`.
- (1.2) — Otherwise, `decay-copy(rend(E.t))` if it is a valid expression and its type `S` models `sentinel_for <decltype(ranges::rbegin(E))>` with overload resolution performed in a context that includes the declaration:
- ```
template<class T> void rend(T&&) = delete;
```
- and does not include a declaration of `ranges::rend`.
- (1.3) — Otherwise, `make_reverse_iterator(ranges::begin(E.t))` if both `ranges::begin(E.t)` and `ranges::end(E.t)` are valid expressions of the same type `I` which models `bidirectional_iterator ([iterator.concept.bidir])`.
- (1.4) — Otherwise, `ranges::rend(E)` is ill-formed. [ Note: This case can result in substitution failure when `ranges::rend(E)` appears in the immediate context of a template instantiation. — *end note* ]
- 2 [ Note: Whenever `ranges::rend(E)` is a valid expression, the types `S` and `I` of `ranges::rend(E)` and `ranges::rbegin(E)` model `sentinel_for <S, I>`. — *end note* ]

Change 24.4.2 [range.range]:

- 1 The range concept defines the requirements of a type that allows iteration over its elements by providing an iterator and sentinel that denote the elements of the range.

```
- template<class T>
-   concept range-impl = // exposition only
-       requires(T&& t) {
-           ranges::begin(std::forward<T>(t)); // sometimes equality-
preserving (see below)
-           ranges::end(std::forward<T>(t));
-       };
-
- template<class T>
-   concept range = range-impl<T&&>;
-
- template<class T>
```

```

- concept forwarding-range = // exposition only
- range<T> && range-impl<T>;

+ template<class T>
+ concept range =
+ requires(T& t) {
+ ranges::begin(t); // sometimes equality-
preserving (see below)
+ ranges::end(t);
+ };
+
+ template<class T>
+ concept safe_range =
+ range<T> &&
+ (is_lvalue_reference_v<T> || enable_safe_range<remove_cvref_t<T>>);

```

- 5 Given an expression E such that `decltype ((E))` is T ~~and an lvalue t that denotes the same object as E~~, T models ~~forwarding~~ ~~-range~~ `safe_range` only if the validity of iterators obtained from the object denoted by E is not tied to the lifetime of that object.

- (5.1) — ~~ranges::begin(E) and ranges::begin(t) are expression-equivalent,~~
- (5.2) — ~~ranges::end(E) and ranges::end(t) are expression-equivalent, and~~
- (5.3) — ~~the validity of iterators obtained from the object denoted by E is not tied to the lifetime of that object.~~

- 6 [ Note: Since the validity of iterators is not tied to the lifetime of an object whose type models ~~forwarding~~ ~~-range~~ `safe_range`, a function can accept arguments of such a type by value and return iterators obtained from it without danger of dangling. — end note ]

```

+ template<class>
+ inline constexpr bool enable_safe_range = false;

```

- 6\* Remarks: Pursuant to [namespace.std], users may specialize `enable_safe_range` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions ([expr.const]) and have type `const bool`.

- 7 [ Example: Specializations of class template `subrange` model ~~forwarding~~ ~~-range~~ `safe_range`. `subrange` ~~provides non-member rvalue overloads of begin and end with the same semantics as its member lvalue overloads~~ specializes `enable_safe_range` to `true`, and `subrange`'s iterators - since they are “borrowed” from some other range - do not have validity tied to the lifetime of a `subrange` object. — end example ]

Change 24.4.5 [range.refinements], the definition of the `viewable_range` concept:

- 4 The `viewable_range` concept specifies the requirements of a range type that can be converted to a view safely.

```

template<class T>
concept viewable_range =
- range<T> && (forwarding-range<T> || view<decay_t<T>>);
+ range<T> && (safe_range<T> || view<decay_t<T>>);

```

Change 24.5.3 [range.subrange], to use `safe_range` instead of ~~forwarding~~ ~~-range~~, to remove the non-member `begin/end` overloads that were the old opt-in, and to add a specialization for `enable_safe_range` which is

the new opt-in:

- 1 The subrange class template combines together an iterator and a sentinel into a single object that models the view concept. Additionally, it models the sized\_range concept when the final template parameter is subrange\_kind ::sized.

```
namespace std::ranges {

    template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind K
=
    sized_sentinel_for<S, I> ? subrange_kind::sized :
subrange_kind::unsized>
    requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
    class subrange : public view_interface<subrange<I, S, K>> {

        template<not-same-as<subrange> R>
-         requires forwarding_range<R> &&
+         requires safe_range<R> &&
            convertible_to<iterator_t<R>, I> && convertible_to<sentinel_t<R>, S>
            constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);

-         template<forwarding_range R>
+         template<safe_range R>
            requires convertible_to<iterator_t<R>, I> &&
convertible_to<sentinel_t<R>, S>
            constexpr subrange(R&& r, make_unsigned_like_t<iter_difference_t<I>> n)
            requires (K == subrange_kind::sized)
                : subrange{ranges::begin(r), ranges::end(r), n}
            {}

-         friend constexpr I begin(subrange&& r) { return r.begin(); }
-         friend constexpr S end(subrange&& r) { return r.end(); }
        };

-         template<forwarding_range R>
+         template<safe_range R>
            subrange(R&&) ->
                subrange<iterator_t<R>, sentinel_t<R>,
                    (sized_range<R> || sized_sentinel_for<sentinel_t<R>,
iterator_t<R>>>)
                    ? subrange_kind::sized : subrange_kind::unsized>;

-         template<forwarding_range R>
+         template<safe_range R>
            subrange(R&&, make_unsigned_like_t<range_difference_t<R>>)) ->
                subrange<iterator_t<R>, sentinel_t<R>, subrange_kind::sized>;

        template<size_t N, class I, class S, subrange_kind K>
            requires (N < 2)
            constexpr auto get(const subrange<I, S, K>& r);
    }
}
```

Change the name of the concept in 24.5.3.1 [range.subrange.ctor]:

```
template<not-same-as<subrange> R>
-     requires forwarding_range<R> &&
+     requires safe_range<R> &&
        convertible_to<iterator_t<R>, I> && convertible_to<sentinel_t<R>,
```

```
S>
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);
```

Change the name of the concept in 24.5.4 [range.dangling]:

- 1 The tag type `dangling` is used together with the template aliases `safe_iterator_t` and `safe_subrange_t` to indicate that an algorithm that typically returns an iterator into or subrange of a range argument does not return an iterator or subrange which could potentially reference a range whose lifetime has ended for a particular rvalue range argument which does not model ~~*forwarding*~~ *safe\_range* ~~safe\_range~~ `safe_range` ([range.range]).

- 2 [ *Example*: [...]

The call to `ranges::find` at #1 returns `ranges::dangling` since `f()` is an rvalue vector; the vector could potentially be destroyed before a returned iterator is dereferenced. However, the calls at #2 and #3 both return iterators since the lvalue `vec` and specializations of `subrange` model ~~*forwarding*~~ *safe\_range* ~~safe\_range~~ `safe_range`. — *end example* ]

Remove the non-member old opt-ins in 24.6.1.2 [range.empty.view]:

```
namespace std::ranges {
    template<class T>
        requires is_object_v<T>
        class empty_view : public view_interface<empty_view<T>> {
        public:

-     friend constexpr T* begin(empty_view) noexcept { return nullptr; }
-     friend constexpr T* end(empty_view) noexcept { return nullptr; }
        };
}
```

Remove the non-member old opt-ins in 24.7.3.1 [range.ref.view]:

```
namespace std::ranges {
    template<range R>
        requires is_object_v<R>
        class ref_view : public view_interface<ref_view<R>> {
        private:
            R* r_ = nullptr;           // exposition only
        public:
-     friend constexpr iterator_t<R> begin(ref_view r)
-     { return r.begin(); }

-     friend constexpr sentinel_t<R> end(ref_view r)
-     { return r.end(); }
        };
    template<class R>
        ref_view(R&) -> ref_view<R>;
}
```

## § 6 Acknowledgements

Thanks to Eric Niebler and Casey Carter for going over this paper with me, and correcting some serious misconceptions earlier drafts had. Thanks to Tim Song and Agustín Bergé for going over the details. Thanks to Tony van Eerd for helping with naming.

## § 7 References

---

[LWG3247] Casey Carter. `ranges::iter_move` should perform ADL-only lookup of `iter_move`.

<https://wg21.link/lwg3247>

[msvc.basic\_string\_view] non-member `begin` `()/end` `()` for `basic_string_view`.

<https://github.com/microsoft/STL/blame/92508bed6387cbdae433fc86279bc446af6f1b1a/stl/inc/xstring#L1207-L1216>

[P0896R1] Eric Niebler, Casey Carter. 2018. Merging the Ranges TS.

<https://wg21.link/p0896r1>

[P0970R1] Eric Niebler. 2018. Better, Safer Range Access Customization Points.

<https://wg21.link/p0970r1>

[P1870R0] Barry Revzin. 2019. forwarding-range is too subtle.

<https://wg21.link/p1870r0>

[P1871R0] Barry Revzin. 2019. Should concepts be enabled or disabled?

<https://wg21.link/p1871r0>

[P1900R0] Barry Revzin. 2019. Concepts-adjacent problems.

<https://wg21.link/p1900r0>

[std.429] Casey Carter. 2018. Consider removing support for rvalue ranges from range access CPOs.

<https://github.com/ericniebler/stl2/issues/429>

[std.547] Eric Niebler. 2018. Redesign `begin/end` CPOs to eliminate deprecated behavior.

<https://github.com/ericniebler/stl2/issues/547>

[std.592] Eric Niebler. 2018. `const` `subrange` `<I, S,` `[un` `]sized` `>` is not a *forwarding* *-range*.

<https://github.com/ericniebler/stl2/issues/592>

---

1. There is a hypothetical kind of range where the range itself owns its data by `shared_ptr`, and the iterators *also* share ownership of the data. In this way, the iterators' validity isn't tied to the range's lifetime not because the range doesn't own the elements (as in the `span` case) but because the iterators *also* own the elements. I'm not sure if anybody has ever written such a thing.↵
2. I intend this as a positive, not as being derogatory.↵